PHASE 4 — THE ENTERPRISE CRYPTOGRAPHIC ECOSYSTEM

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 4 of 22

Document Type: Technical Research & Engineering Handbook

Status: Architecture Foundation

Table of Contents

12. Java Cryptography Architecture
13. .NET Cryptography
14. Python Cryptography
15. Go Cryptography
16. Rust Cryptography
17. Node.js Cryptography
18. Native and System Libraries
19. Static and Dynamic Linking
20. Package Managers and Dependencies
21. Source Repositories
22. Configuration Files
23. Environment Variables
24. Secrets and Key Material
25. Digital Certificates
26. TLS Infrastructure
27. SSH Infrastructure
28. VPN Infrastructure
29. PKI Infrastructure
30. Hardware Security Modules
31. Cloud Key Management
32. Cloud Certificate Services
33. Databases

34. Storage Systems
35. Backups and Archives
36. API Gateways
37. Service Meshes
38. Containers
39. Docker Images
40. Kubernetes
41. CI/CD Pipelines
42. Software Supply Chain
43. Code Signing
44. Firmware
45. IoT and Embedded Systems
46. Mobile Applications
47. Blockchain Systems
48. Third-Party and SaaS Dependencies
49. Legacy Systems
50. Network Infrastructure
51. Cryptographic Discovery Challenges
52. Direct vs Indirect Cryptographic Usage
53. Explicit vs Implicit Cryptography
54. Static vs Runtime Discovery
55. Building the Enterprise Crypto Map

---

1. Purpose of This Phase

The first three phases established:

```
Phase 1
Cryptography
        ↓
Phase 2
Quantum Threat
        ↓
```

```
    Phase 3
    Post-Quantum Cryptography
```

Now we face the real engineering problem:

> **Where is all this cryptography actually being used?**

An enterprise does not have one cryptographic system.

It may have thousands.

Cryptography exists inside:

- Source code
- Libraries
- Containers
- Operating systems
- Network protocols
- Certificates
- Cloud services
- Hardware
- Databases
- Mobile applications
- Firmware
- Third-party software
- CI/CD infrastructure

Therefore, ECDAT cannot be a simple source-code scanner.

It must become an **enterprise-wide cryptographic discovery system**.

---

2. The Central Problem: Cryptography Is Everywhere

Consider a hypothetical organization.

```
Enterprise
│
├── 3,000 Repositories
├── 15,000 Applications
├── 40,000 Containers
├── 200,000 Certificates
├── 100,000 Keys
├── 15 Cloud Accounts
├── 5,000 Servers
├── 10,000 Network Devices
├── 2,000 APIs
└── 50,000 Employees
```

Cryptographic operations may exist in every layer.

A developer might write:

```
AES.new(key, AES.MODE_GCM)
```

A Java service might use:

```
Cipher.getInstance("AES/GCM/NoPadding");
```

A server might automatically negotiate:

```
TLS 1.3
ECDHE
AES-256-GCM
```

A cloud service might use:

```
AWS KMS
```

A Kubernetes cluster might use:

```
TLS certificates
```

A firmware image might contain:

```
RSA public key
```

The cryptography may not be visible from a single place.

That is the core discovery challenge.

---

3. What Is a Cryptographic Asset?

ECDAT needs a precise definition.

A **cryptographic asset** is any software, hardware, configuration, key, certificate, protocol, dependency, service, or other resource that performs, stores, references, manages, or depends upon cryptographic operations.

Examples:

```
Algorithm
Key
Certificate
Library
API
Protocol
HSM
KMS
Crypto Provider
Configuration
Firmware
Container
Application
```

---

4. Cryptographic Asset Taxonomy

ECDAT can organize assets into major categories.

```
Cryptographic Assets
│
├── Algorithms
│
├── Keys
│
├── Certificates
│
├── Protocols
```

```
      │
      ├── Libraries
      │
      ├── APIs
      │
      ├── Applications
      │
      ├── Hardware
```

Each asset can then have relationships.

For example:

```
Application
   │
   ├── uses → OpenSSL
   │
   ├── uses → RSA
   │
   ├── presents → Certificate
   │
   └── protects → Customer Data
```

---

5. Source Code

Source code is one of the most important discovery targets.

Developers may directly invoke cryptographic APIs.

Example:

```
from cryptography.hazmat.primitives.ciphers import Cipher
```

Or:

```
crypto.createCipheriv(...)
```

Or:

```
Cipher.getInstance(...)
```

Or:

```
aes.NewCipher(...)
```

ECDAT should detect these patterns.

But this is only the beginning.

---

6. Programming Languages

Enterprise environments use many languages.

Potentially relevant languages include:

```
Python
Java
JavaScript
TypeScript
```

```
C
C++
C#
Go
Rust
Swift
Kotlin
PHP
Ruby
Dart
Scala
Objective-C
```

Each language has different cryptographic libraries and APIs.

Therefore, ECDAT needs language-specific detection logic.

---

7. Cryptographic APIs

Applications rarely implement cryptography mathematically from scratch.

Instead they use APIs.

Example:

```
Application
    ↓
Crypto API
    ↓
```

```
Crypto Library
    ↓
Algorithm
```

For example:

```
Java Application
       ↓
JCA/JCE
       ↓
Provider
       ↓
OpenSSL / Native Crypto
```

This creates a dependency chain.

ECDAT needs to understand the entire chain.

---

8. Cryptographic Libraries

Common cryptographic libraries include:

- OpenSSL
- BoringSSL
- LibreSSL
- libsodium
- Bouncy Castle
- Botan

- wolfSSL
- mbed TLS
- Java security providers
- Windows CNG

Libraries are especially important because one vulnerable library can affect hundreds of applications.

---

9. OpenSSL

OpenSSL is one of the most widely deployed cryptographic libraries.

It supports:

- TLS
- X.509
- RSA
- ECC
- AES
- SHA
- HMAC
- Certificates
- Key management

ECDAT should identify:

```
OpenSSL
Version
Build
```

```
Configuration
Applications
Algorithms
```

For example:

```
OpenSSL 3.x
        ↓
TLS
        ↓
ECDHE
        ↓
AES-GCM
```

A library-level dependency may be more important than a single source-code occurrence.

---

10. BoringSSL

BoringSSL is Google's fork of OpenSSL used in major systems.

It is used in areas including:

- Chrome
- Android
- Google infrastructure

ECDAT should not assume that:

```
OpenSSL detected
```

means every OpenSSL-like implementation is actually OpenSSL.

Library identity matters.

---

11. LibreSSL

LibreSSL is another OpenSSL-derived project.

It is used in various operating systems and applications.

ECDAT should maintain separate library identities.

---

12. Java Cryptography Architecture

Java applications often use the:

Java Cryptography Architecture (JCA)

and:

Java Cryptography Extension (JCE)

Applications may request:

```
Cipher.getInstance(...)
```

or:

```
Signature.getInstance(...)
```

The actual implementation can depend on the configured provider.

This creates an important discovery problem:

```
Java API
   ↓
Provider
   ↓
Implementation
   ↓
Algorithm
```

A regex scanner may identify `Cipher`, but determining the actual cryptographic primitive may require deeper analysis.

---

13. .NET Cryptography

.NET provides APIs such as:

```
System.Security.Cryptography
```

Examples include:

- RSA
- ECDsa
- ECDiffieHellman
- AES
- SHA
- HMAC

ECDAT should understand both the API and the implementation context.

---

14. Python Cryptography

Python applications may use:

```
cryptography
PyCryptodome
hashlib
ssl
OpenSSL bindings
```

Example:

```
AES.new(...)
```

But a sophisticated scanner must distinguish:

```
Hashing for integrity
```

from:

```
Hashing used as a cryptographic security mechanism
```

Context matters.

---

15. Go Cryptography

Go has a strong standard cryptographic ecosystem.

Examples:

```
crypto/aes
crypto/rsa
crypto/ecdsa
crypto/ed25519
crypto/tls
crypto/sha256
```

Go's standard library makes static discovery relatively structured.

ECDAT should still identify:

- Algorithm
- Key size
- Usage
- TLS context
- Certificate usage

---

16. Rust Cryptography

Rust has multiple cryptographic ecosystems.

Examples include:

```
RustCrypto
ring
rustls
OpenSSL bindings
```

Dependency analysis becomes important because Rust projects may use different cryptographic implementations.

---

17. Node.js Cryptography

Node.js provides:

```
node:crypto
```

Applications may use:

- AES
- RSA
- ECDSA
- Diffie-Hellman
- HMAC
- Hashes
- TLS

ECDAT should detect both direct API calls and dependencies.

---

18. Native and System Libraries

Applications may dynamically link to:

```
libcrypto
libssl
```

or other system cryptographic libraries.

The source code may contain no explicit implementation.

Example:

```
Application
      ↓
Dynamic Linker
      ↓
libcrypto.so
      ↓
OpenSSL
```

Therefore, binary analysis becomes essential.

---

19. Static and Dynamic Linking

There are two major linking models.

Dynamic Linking

```
Application
      ↓
Shared Library
```

The cryptographic implementation exists externally.

Static Linking

```
Application
      ↓
Embedded Library Code
```

The cryptographic implementation may be inside the binary.

This makes discovery harder.

ECDAT therefore needs binary inspection capabilities.

---

20. Package Managers and Dependencies

Modern applications depend heavily on packages.

Examples:

```
npm
pip
Maven
Gradle
NuGet
Cargo
Go Modules
```

A developer may not directly call OpenSSL.

Instead:

```
Application
 ↓
```

```
Framework
 ↓
Dependency
 ↓
Crypto Library
 ↓
Algorithm
```

Therefore, dependency graphs are essential.

---

21. Source Repositories

The NTRO problem specifically mentions scanning source-code repositories.

Possible sources include:

- GitHub
- GitLab
- Bitbucket
- Internal Git servers
- Monorepos
- Archive repositories

ECDAT should support:

```
Repository
↓
Branch
```

```
↓
Commit
↓
Files
↓
Code
↓
Dependencies
↓
Crypto Assets
```

---

22. Configuration Files

Cryptography is often configured rather than coded.

Examples:

```
nginx.conf
application.yml
server.xml
openssl.cnf
ssh_config
```

Configuration may specify:

```
TLS versions
Cipher suites
Certificate paths
```

```
Key paths
Signature algorithms
```

Therefore, configuration scanning is essential.

---

23. Environment Variables

Applications may load cryptographic configuration through environment variables.

Example:

```
TLS_CERT=/certs/server.pem
TLS_KEY=/secrets/server.key
```

or:

```
KMS_KEY_ID=...
```

ECDAT should detect references without exposing sensitive secret values.

---

24. Secrets and Key Material

This is a critical distinction.

ECDAT should discover **cryptographic keys and key references**, but it should not become a secret-exfiltration tool.

For example:

```
PRIVATE_KEY=/vault/prod/key
```

should be represented as:

```
Key Reference:
Vault path

Algorithm:
Unknown

Location:
Production API

Secret Value:
NOT COLLECTED
```

The platform should follow least-privilege principles.

---

25. Digital Certificates

Certificates are among the easiest cryptographic assets to inventory.

ECDAT can extract:

```
Subject
Issuer
Serial Number
Validity
Public Key Algorithm
Key Size
Signature Algorithm
```

```
SAN
Certificate Chain
```

Example:

```
Certificate
│
├── Algorithm: RSA
├── Key Size: 2048
├── Issuer: Example CA
├── Valid Until: ...
└── Application: Payments API
```

---

26. TLS Infrastructure

TLS is one of the largest cryptographic attack surfaces.

ECDAT should identify:

```
TLS Version
Cipher Suite
Key Exchange
Certificate Algorithm
Signature Algorithm
Crypto Library
Server
Endpoint
```

For example:

```
TLS 1.3
+
ECDHE
+
AES-256-GCM
+
ECDSA P-256 Certificate
```

This is a richer asset than simply saying:

```
ECC found
```

---

27. SSH Infrastructure

SSH may use:

- RSA
- ECDSA
- Ed25519
- Diffie-Hellman
- MAC algorithms
- Symmetric encryption

ECDAT should inventory:

```
SSH Server
Host Key
Host Key Algorithm
KEX Algorithm
Cipher
MAC
Configuration
```

---

28. VPN Infrastructure

VPN systems can use cryptography for:

- Authentication
- Key exchange
- Encryption
- Integrity

Examples:

- IPsec
- WireGuard
- OpenVPN
- SSL VPN

ECDAT should identify cryptographic mechanisms at each layer.

---

29. PKI Infrastructure

Enterprise PKI may contain:

```
Root CA
│
├── Intermediate CA
│      │
│      ├── Service Certificates
│      ├── User Certificates
│      └── Device Certificates
│
└── Code Signing CA
```

A change to the cryptographic algorithm can affect the entire hierarchy.

This is why PKI should be represented as a graph.

---

30. Hardware Security Modules

HSMs provide hardware-backed cryptographic protection.

They may store:

- Private keys
- Signing keys
- Encryption keys
- Certificate authority keys

ECDAT should identify:

```
HSM
 |
 ├── Vendor
 ├── Model
 ├── Firmware
 ├── Algorithms
 ├── Key Types
 ├── Applications
 └── PQC Support
```

---

31. Cloud Key Management

Cloud KMS platforms are critical.

Examples include:

- AWS KMS
- Azure Key Vault
- Google Cloud KMS

Applications may never see the underlying cryptographic keys.

Instead:

```
Application
     ↓
Cloud KMS API
     ↓
Managed Key
```

```
        ↓
Cryptographic Operation
```

Therefore, source-code scanning alone cannot discover the complete crypto dependency.

---

32. Cloud Certificate Services

Cloud environments may automatically issue and rotate certificates.

ECDAT needs to understand:

```
Certificate Service
      ↓
Certificate
      ↓
Application
      ↓
Load Balancer
      ↓
TLS
```

This is particularly important because certificate lifecycle can be completely automated.

---

33. Databases

Cryptography may exist at multiple levels.

Application-level encryption

```
Application
↓
Encrypt
↓
Database
```

Database-level encryption

```
Database
↓
Transparent Encryption
```

Storage-level encryption

```
Disk
↓
Encryption
```

ECDAT should distinguish these.

---

34. Storage Systems

Examples:

- Object storage
- File systems
- Block storage

- Backup storage
- Archives

Cryptography may be used for:

- Encryption at rest
- Integrity
- Access control

The important question becomes:

> What algorithm protects the data, and who controls the key?

---

35. Backups and Archives

Long-term archives are especially important for post-quantum risk.

Example:

```
Backup
Retention:
25 years

Encryption:
RSA-based key wrapping

Risk:
Potential HNDL exposure
```

Even if the application itself is modern, old backups may retain vulnerable cryptographic protection.

ECDAT should therefore include historical data stores.

---

36. API Gateways

API gateways frequently terminate TLS.

Architecture:

```
Internet
   ↓
API Gateway
   ↓
Microservice
```

The cryptographic dependency may therefore exist entirely at the gateway.

ECDAT must map:

```
Gateway
↓
Certificate
↓
TLS
↓
Crypto Algorithm
↓
Backend Service
```

---

37. Service Meshes

Modern microservice architectures often use service meshes.

Examples include:

- Istio
- Linkerd

They may automatically provide:

```
mTLS
```

between services.

Therefore, developers may write:

```
No cryptographic code
```

while the system still uses extensive cryptography.

This is a perfect example of why source scanning alone is insufficient.

---

38. Containers

Containers introduce another discovery surface.

An application container may contain:

```
Application
+
Libraries
```

```
+
OS Packages
+
Certificates
+
Crypto Libraries
```

ECDAT should scan:

```
Dockerfile
Image Layers
Packages
Libraries
Binaries
Certificates
Configurations
```

---

39. Docker Images

A Docker image is composed of layers.

Example:

```
Image
│
├── Base OS
├── System Libraries
├── Application Dependencies
```

```
    ├── Application
    └── Configuration
```

A cryptographic library may exist in the base image even if the application doesn't explicitly install it.

ECDAT should therefore track dependency provenance.

---

40. Kubernetes

Kubernetes introduces:

- Secrets
- ConfigMaps
- Ingress
- Service Mesh
- TLS
- Certificates
- Controllers
- Admission systems

A Kubernetes cluster can contain enormous numbers of cryptographic dependencies.

ECDAT should understand:

```
Cluster
↓
Namespace
↓
Workload
```

```
↓
Service
↓
Ingress
↓
Certificate
↓
Crypto Algorithm
```

---

41. CI/CD Pipelines

Cryptography also exists in build systems.

Examples:

```
Git signing
SSH keys
TLS
Package signing
Artifact signing
Secrets
Cloud credentials
```

A CI/CD pipeline may therefore have cryptographic dependencies that are invisible from application repositories.

---

42. Software Supply Chain

The software supply chain includes:

```
Developer
↓
Repository
↓
Build
↓
Dependency Resolution
↓
Artifact
↓
Signing
↓
Registry
↓
Deployment
```

Each stage can involve cryptography.

ECDAT should eventually connect these stages.

---

43. Code Signing

Code signing is particularly sensitive.

A private signing key allows an organization to establish trust in software artifacts.

If a signing system uses quantum-vulnerable public-key cryptography, migration becomes strategically important.

Potential assets:

```
Signing Key
Certificate
Signing Algorithm
Build Server
Signing Service
HSM
Artifact Registry
```

---

44. Firmware

Firmware can contain:

- Public keys
- Signature verification code
- Encryption
- Secure boot logic

For example:

```
Boot ROM
   ↓
Bootloader
   ↓
Firmware Signature
   ↓
Verify Public Key
```

```
        ↓
    Run Firmware
```

If the trust anchor is quantum-vulnerable, future migration can be difficult.

---

45. IoT and Embedded Systems

IoT systems may use:

- Device certificates
- Secure boot
- TLS
- Firmware signatures
- Device keys

Challenges include:

- Limited CPU
- Limited memory
- Long deployment lifetime
- Infrequent updates
- Physical deployment

These factors increase migration complexity.

---

46. Mobile Applications

Mobile applications may include:

- Certificate pinning
- TLS
- Local encryption
- Secure storage
- Digital signatures
- Cryptographic libraries

ECDAT could scan:

```
APK
IPA
Native Libraries
Embedded Certificates
Embedded Keys
Crypto APIs
```

---

47. Blockchain Systems

Blockchain systems may use:

- Digital signatures
- Hash functions
- Merkle trees
- Consensus cryptography
- Wallet keys

ECDAT could classify:

```
Blockchain
│
├── Signature Scheme
├── Hash Function
├── Wallet Keys
├── Smart Contracts
└── Cryptographic Dependencies
```

This is particularly relevant to the SIH theme.

---

48. Third-Party and SaaS Dependencies

An organization may rely on:

```
Payment Gateway
Identity Provider
Cloud Provider
SaaS Platform
External API
```

The organization may not control the underlying cryptography.

ECDAT should therefore distinguish:

```
Owned Crypto
```

from:

```
Third-Party Crypto
```

and:

```
Unknown Crypto
```

This becomes important for migration planning.

---

49. Legacy Systems

Legacy systems represent some of the hardest migration problems.

Example:

```
Mainframe
   ↓
Legacy Application
   ↓
RSA
   ↓
Custom Hardware
   ↓
No Modern API
```

Such systems may have:

- undocumented cryptography
- custom implementations
- unsupported libraries

- hardcoded keys
- no source code
- long replacement cycles

ECDAT should assign higher uncertainty and migration complexity to these systems.

---

50. Network Infrastructure

Network devices can contain:

- TLS
- SSH
- IPsec
- Certificates
- Device keys
- Firmware signatures

Examples:

- Routers
- Firewalls
- Load balancers
- Proxies
- IDS/IPS
- VPN gateways

This expands ECDAT beyond application security into infrastructure security.

---

51. Cryptographic Discovery Challenges

Now we reach one of the most important engineering sections.

Finding cryptography is surprisingly difficult.

Why?

Because cryptographic usage exists in several forms.

---

52. Direct vs Indirect Cryptographic Usage

Direct

```
AES.new(...)
```

Easy to detect.

Indirect

```
secure_store.encrypt(...)
```

The actual algorithm is hidden behind another abstraction.

Deep Indirect

```
Application
↓
Framework
```

```
↓
Dependency
↓
Provider
↓
Library
↓
Algorithm
```

This requires dependency analysis and semantic reasoning.

---

53. Explicit vs Implicit Cryptography

Explicit:

```
RSA
ECDSA
AES
```

Implicit:

```
HTTPS
mTLS
SSH
KMS
Secure Boot
Signed Package
```

The system may never mention "RSA" directly.

Yet cryptography is still present.

Therefore, ECDAT should discover **cryptographic behavior**, not merely cryptographic keywords.

---

54. Static vs Runtime Discovery

Static analysis examines:

```
Source Code
Binary
Container
Configuration
Dependencies
```

Runtime analysis examines:

```
Live Traffic
Processes
System Calls
TLS Handshakes
Loaded Libraries
```

Both provide different information.

Static Analysis

Advantages:

- Safe
- Repeatable
- Broad
- Works before deployment

Limitations:

- May miss dynamic behavior
- May overestimate unused dependencies

Runtime Analysis

Advantages:

- Shows actual behavior
- Detects dynamically loaded libraries
- Captures real protocol usage

Limitations:

- Requires deployment access
- Can miss rarely executed code
- More operationally complex

---

55. Building the Enterprise Crypto Map

ECDAT should construct a global model.

```
Enterprise
│
├── Business Unit
│      │
│      └── Application
│             │
│             ├── Repository
│             ├── Container
│             ├── Library
│             ├── Certificate
│             └── Crypto Algorithm
│
├── Infrastructure
│      │
│      ├── Server
│      ├── Firewall
│      ├── VPN
│      └── HSM
```

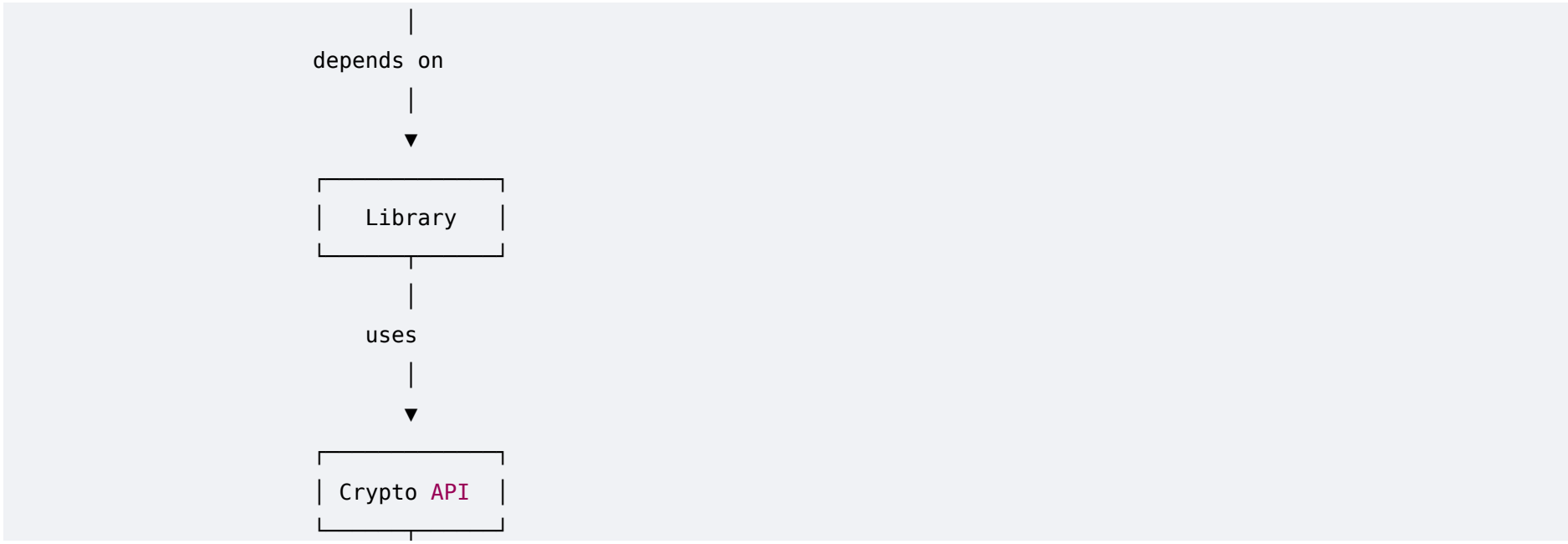
This becomes the foundation for the enterprise cryptographic inventory.

---

56. Cryptographic Dependency Graph

A graph representation could look like:

```
                    ┌─────────────┐
                    │ Application │
                    └──────┬──────┘
```



This graph becomes incredibly powerful when combined with quantum-risk information.

---

57. ECDAT Asset Model

A cryptographic asset could be represented as:

```
{
  "asset_id": "CRYPTO-001928",
  "asset_type": "algorithm",
  "algorithm": "ECDSA",
  "parameter_set": "P-256",
  "function": "digital_signature",
  "location": "payments-service",
```

```
        "repository": "payments-api",
        "library": "OpenSSL",
        "library_version": "3.x",
        "protocol": "TLS",
        "business_criticality": "critical",
        "data_classification": "restricted",
        "data_lifetime_years": 15,
        "quantum_status": "vulnerable",
        "migration_complexity": "high",
        "confidence": 0.96
```

This is the type of normalized record that could eventually feed the CBOM.

---

58. Discovery Confidence

Every discovery should have a confidence value.

Example:

```
Direct API Match
Confidence = 0.99
```

```
Binary Signature Match
Confidence = 0.93
```

```
Dependency Inference
Confidence = 0.82
```

```
LLM Semantic Inference
Confidence = 0.71
```

This prevents ECDAT from presenting uncertain findings as absolute facts.

---

59. Asset Ownership

Finding an asset is not enough.

The organization needs to know:

> Who is responsible for it?

Therefore:

```
Crypto Asset
      ↓
Application
      ↓
Team
      ↓
Business Owner
      ↓
Security Owner
```

This enables actionable migration planning.

Instead of:

> "4,000 RSA assets detected."

ECDAT should say:

> "Team Payments owns 381 high-priority RSA dependencies."

That is significantly more useful.

---

60. Data Classification

ECDAT should ideally associate cryptographic assets with data classifications.

For example:

```
Public
Internal
Confidential
Restricted
Highly Restricted
```

The exact taxonomy should be configurable.

This information directly affects quantum risk.

---

61. Business Context

Technical cryptography cannot be evaluated in isolation.

Consider:

```
Algorithm:
RSA-2048
```

This tells us very little.

Now add:

```
Application:
National Identity Service

Data:
Identity Records

Retention:
30 years

Business Criticality:
Critical

Exposure:
Internet

Migration:
5 years
```

Now ECDAT can produce a meaningful risk assessment.

---

62. Enterprise Crypto Inventory

The final inventory should look conceptually like:

Asset	Type	Algorithm	Location	Owner	Criticality	Quantum Risk
CRYPTO-001	Certificate	RSA-2048	API Gateway	Payments	Critical	High
CRYPTO-002	Key Exchange	ECDH P-256	VPN	Network	High	Critical
CRYPTO-003	Encryption	AES-256	Database	Data	Critical	Lower
CRYPTO-004	Signature	ECDSA P-256	CI/CD	DevOps	High	Critical
CRYPTO-005	Hash	SHA-256	Repository	Platform	Medium	Context-dependent

This is the beginning of the **Cryptographic Bill of Materials**.

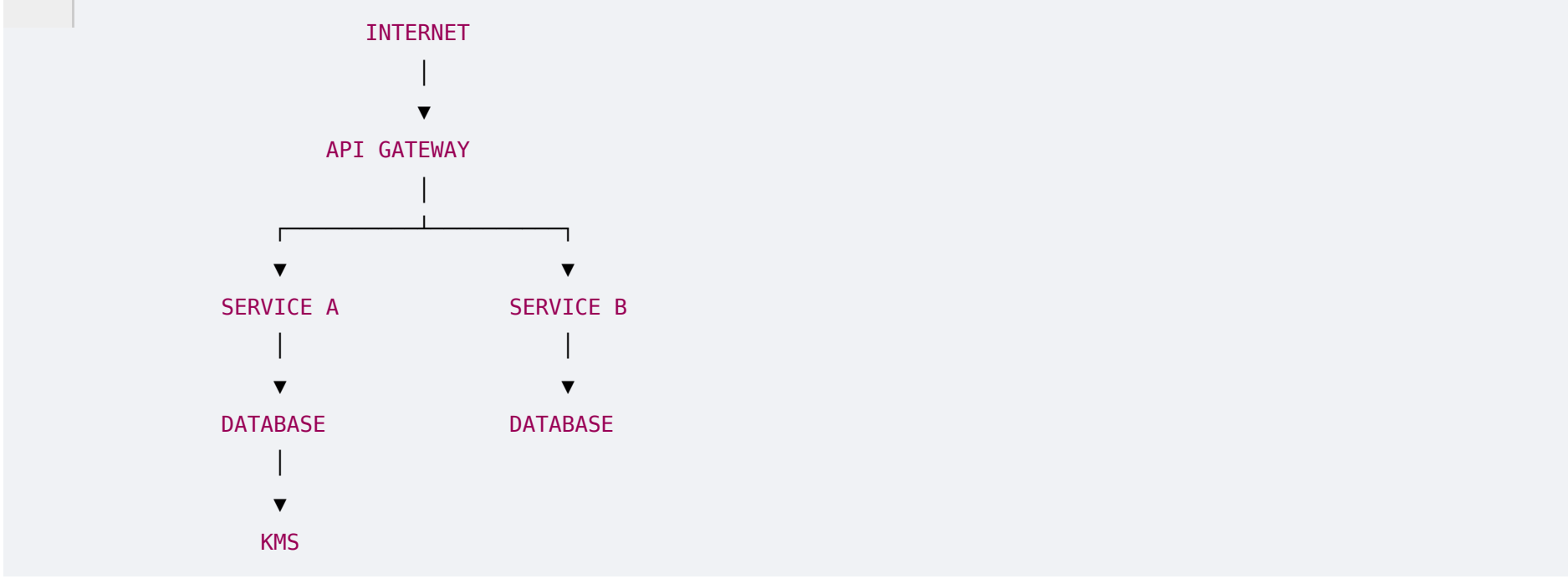
---

63. Example Enterprise

Let's imagine:

```
NATIONAL DIGITAL SERVICES
```

Infrastructure:



Additional infrastructure:

```
Cloud
|
├── Kubernetes
├── Load Balancers
├── KMS
```

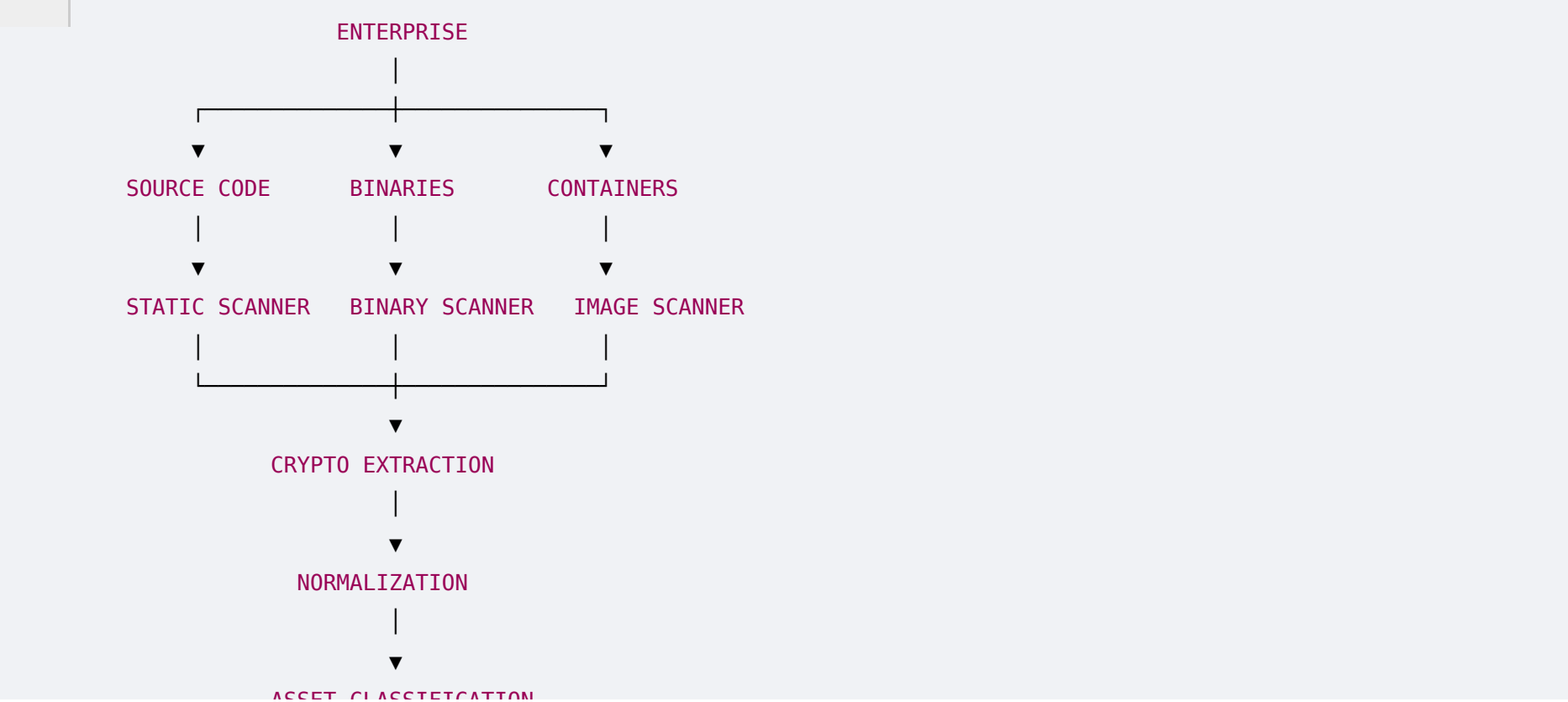
```
      ├── Certificates
      └── Secrets Manager
```

ECDAT should discover cryptography across the entire architecture.

---

64. ECDAT Discovery Pipeline

The enterprise discovery architecture can now be defined:



This architecture will become increasingly detailed in later phases.

65. Requirements Derived From This Phase

Phase 4 produces a major set of requirements.

R1 — Multi-source discovery

ECDAT must support multiple asset sources.

R2 — Multi-language source analysis

At minimum, the architecture should be extensible for major enterprise languages.

R3 — Library identification

The system must identify cryptographic libraries and versions.

R4 — Binary analysis

ECDAT must inspect binaries and linked libraries.

R5 — Container analysis

ECDAT must scan container images and layers.

R6 — Configuration analysis

Cryptographic settings must be discovered.

R7 — Certificate analysis

Certificates must be parsed and classified.

R8 — Cloud integration

Cloud KMS and certificate systems should be discoverable.

R9 — Runtime visibility

The architecture should eventually support runtime telemetry.

R10 — Dependency mapping

Cryptographic assets must be linked to applications and business owners.

R11 — Confidence scoring

Every finding should carry evidence and confidence.

R12 — No secret exfiltration

ECDAT must distinguish between:

```
Discovering a key exists
```

and:

```
Collecting the key itself
```

The latter should generally be avoided unless explicitly required and securely authorized.

---

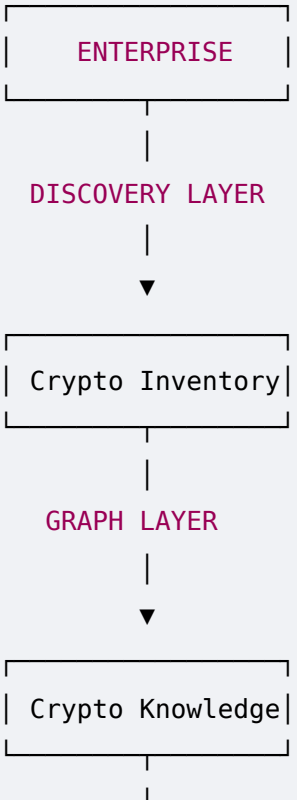
66. The Most Important Architectural Insight

After Phase 4, the ECDAT problem becomes much clearer.

The system isn't simply:

```
Scanner
```

It is:



---

67. Phase 4 Summary

The enterprise cryptographic ecosystem is enormous.

Cryptography can exist in:

- Applications
- Source code
- Libraries
- Binaries
- Containers
- Kubernetes
- Certificates
- TLS
- SSH
- VPN
- PKI
- HSMs
- Cloud KMS
- Databases
- Backups
- APIs
- Service meshes
- CI/CD

- Code signing
- Firmware
- IoT
- Mobile applications
- Blockchain
- Third-party systems
- Legacy infrastructure

Therefore, ECDAT must discover **cryptographic behavior across the entire enterprise**, not merely search source code for algorithm names.

The key transition is:

```
Find Cryptography
       ↓
Understand Cryptography
       ↓
Map Dependencies
       ↓
Understand Business Context
       ↓
Assess Quantum Risk
       ↓
Recommend Migration
```

---

68. The Core Insight

The most important conclusion from this phase is:

> **Cryptographic discovery is an asset-management problem, a dependency-analysis problem, a cybersecurity problem, and ultimately a business-context problem.**

A simple scanner can tell an organization:

> "RSA exists."

A true ECDAT platform should be able to tell it:

> "RSA-2048 is used by the production payments gateway through OpenSSL, protects an internet-facing critical service, is associated with data retained for 15 years, depends on three infrastructure components, has a high migration complexity, and should be prioritized for PQC migration."

That is the difference between a **scanner** and a **cryptographic intelligence platform**.

---

69. Phase 5 Preview — Cryptographic Discovery & CBOM

Now we have reached one of the most important phases of the entire project.

Phase 5 will turn the enterprise ecosystem into an actual **discovery and inventory system**.

We will go deep into:

- What CBOM actually is
- CBOM vs SBOM
- Cryptographic asset schemas
- CycloneDX
- SPDX

- CryptoBOM concepts
- Algorithm detection
- API detection
- AST analysis
- Regex detection
- Dependency analysis
- Binary analysis
- Container analysis
- Certificate parsing
- Protocol discovery
- Library fingerprinting
- YARA-style detection
- Semgrep
- Tree-sitter
- CodeQL
- Trivy
- Syft
- Grype
- OpenSSL inspection
- Binary reverse engineering
- Evidence collection
- False positives

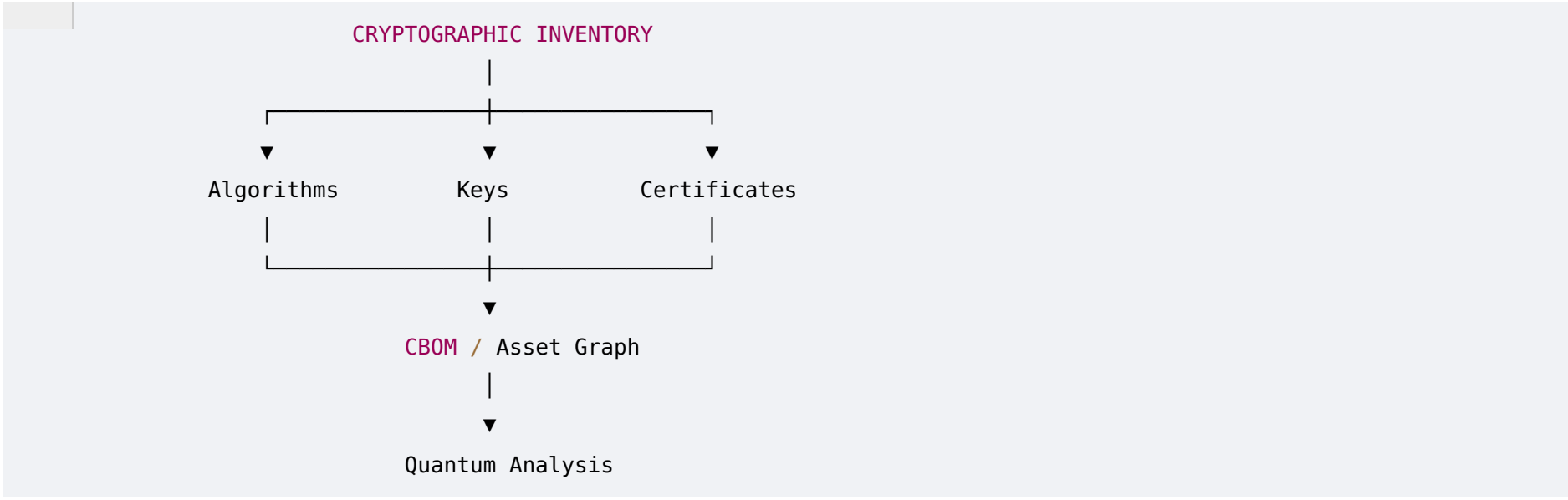
- False negatives
- Confidence scoring
- Asset deduplication
- Version tracking
- Crypto lineage
- CBOM generation
- CBOM storage
- CBOM visualization
- Incremental scanning
- Continuous discovery
- Git integration
- CI/CD integration

Most importantly, we'll design the **actual ECDAT discovery engine** that can take:

```
Repository
     +
Binary
     +
Library
     +
Container
     +
Certificate
```

```
        +
    Configuration
```

and transform it into:



Phase 4 complete.